Shipping-Document Extraction Challenge

Background

Freight moves on paperwork. Every international shipment comes with a **commercial invoice** (what was sold: who sold it, who bought it, and which parts in what quantities) and an **air waybill** (how it travels: carrier, waybill numbers, airports, pieces and weight). These documents arrive as PDFs, phone photos and fax-quality scans, in hundreds of layouts, one per supplier. Reading them into structured data is still largely manual.

Your task is to build a system that reads these document images and returns structured JSON, and that knows when it cannot read a value.

> ***All data in this challenge is synthetic.*** *Every company, person, address, part number and reference number is invented. Cities and airport codes are real, public geography. Any resemblance to a real organisation is coincidental.*

The data

```
data/
  train/  images/  labels/     400 documents with labels
  dev/    images/  labels/     100 documents with labels (same suppliers as train)
  test/   images/              200 documents, no labels (scored by us)
```

- Each document has one or more page images: `<doc_id>_p1.png`, `<doc_id>_p2.png`, and so on. Scanned pages are `.jpg`.
- About 80% are invoices and 20% air waybills.
- Documents come from about 20 fictional suppliers, and each supplier has its own layout: column order, header wording, fonts, date format, part-number style.
- About 40% of pages are **scanned**, with rotation, noise, blur and compression.
- About 40% of invoices run to **multiple pages**. On page 2 the item table may or may not repeat its header row.
- Item tables range from 1 to 30 rows. The same part number can appear on several rows.
- Some documents have a field that is **illegible** (smudged or cut off). Its true value is `null`.
- **The test set includes suppliers that appear nowhere in train or dev.** About half the test documents use layouts you have never seen.

What to extract

See `schema.json` for the exact format and `data/train/labels/` for examples.

CONFIDENTIAL · © 2026 FourKites, Inc. · Proprietary. For the named recipient's interview evaluation only; do not distribute.

Document	Header fields	Line items (every row of the item table)
invoice	invoice_number, invoice_date (ISO), supplier_name, buyer_name, ship_to_name, currency (ISO 4217), total_amount, awb_number	supplier_part_number, customer_part_number, purchase_order, quantity
waybill	carrier, mawb, hawb, origin_airport, destination_airport (IATA), shipper_name, consignee_name, pieces, gross_weight_kg	none

Rules:

- Output **exactly** what is on the page. Don't correct, complete or infer values.
- If a value is missing or illegible, output `null`. **A guessed value scores as wrong**, and false fills are reported separately.
- Line items may be in any order, but every row counts, including repeats.

Scoring

```
python3 score.py --gold data/dev/labels --pred your_dev_predictions.json
```

`score.py` is the scorer we use on the hidden test set.

```
OVERALL = 0.4 x header field accuracy
        + 0.4 x line-item row F1        (a row is right only if all four fields are right)
        + 0.2 x documents fully correct
```

We also report results by slice: scanned vs digital, seen vs **unseen suppliers**, multi-page, repeated parts and illegible fields. We look closely at unseen suppliers and at the false-fill rate. A system that is 90% right and knows which 10% it's unsure of is worth more to us than one that is 93% right and can't tell.

What to submit

1. `test_predictions.json`: predictions for all 200 test documents, in the format of `sample_submission.json`.
2. `report.pdf` (2 pages at most):
   - your approach and why you chose it
   - your dev results by slice (scanned vs digital, seen vs unseen supplier, multi-page, repeated parts, illegible), and where it fails
   - how you would handle a new supplier layout arriving tomorrow
   - **(optional)** a per-field confidence or "needs human review" flag, with its precision and recall on dev
   - the GitHub, Hugging Face and Weights & Biases links (below), so we can reproduce it
3. **A GitHub repository link** with all your code and instructions to reproduce the predictions from the images: seeded and documented, so we can re-run it end to end.

CONFIDENTIAL · © 2026 FourKites, Inc. · Proprietary. For the named recipient's interview evaluation only; do not distribute.

4. **Hugging Face Hub link(s)** to any model you train or fine-tune, with a short model card (training data, intended use, metrics, limitations) so we can load and run it. If your approach trains nothing (pure OCR plus rules), state that; this is then not required.
5. **A Weights & Biases run link** if you train or fine-tune any component, logging the relevant metrics: training and eval loss curves, dev `OVERALL` and its slice breakdown, and the run config / hyper-parameters. If nothing is trained, this is not required.

Make the GitHub repo, Hugging Face model and W&B project **public**, or share access so we can open them, and include all links in `report.pdf`.

Constraints

- Any open-weight model, library or technique is allowed, including fine-tuning, OCR + rules, and vision-language models.
- Inference must run on a single GPU with 24 GB of memory or less, or on CPU. **No paid hosted APIs at inference time.**
- Don't label the test images by hand.
- Suggested time: about 6–8 hours. We value a clear, honest analysis over a leaderboard number.

What we look for

- correctness on unseen layouts, not just memorised ones
- honest handling of uncertainty (nulls and review flags)
- clean, reproducible code
- clear reasoning about failure modes and trade-offs

CONFIDENTIAL · © 2026 FourKites, Inc. · Proprietary. For the named recipient's interview evaluation only; do not distribute.